Udyam Digital - solution architecture and data flows

Red outline = deterministic business logic (source of truth) · Blue = AI (interprets and phrases only) · Grey = data · Dashed = external service · Numbers = data flows, listed below

CLIENT · Next.js 16, React, TypeScript, Tailwind · mobile-first · 10 languages · voice in and out

Onboarding + advisor chat
profile, GSTIN check, chat, voice

Bundle + quote
toggles, live repricing, fixes

Checkout + tracker
UPI, net banking, card, slots

Smart KYC
consent, upload, fix list

Admin dashboard
staff: funnel, KYC queue, leads

Next.js rewrite proxy /api/* · no keys or backend URL in browser code · JWT, rate limits and validation at the edge

BACKEND · Python FastAPI, SQLAlchemy, Pandas · JWT roles (customer, admin) · validation · rate limits · masked logs

REST routers: auth · journeys and advisor · recommendations and quotes · orders and payments · KYC · admin analytics · mock registries

Advisor (AI + rules)
Reads needs, rejections, amounts
At most 5 questions, no repeats
Gemini, Claude or offline rules
Voice: browser speech or Gemini
Guard drops any reply with a ₹ amount or product the engine did not produce

Catalogue and pricing
Eligibility: tier, size, GSTIN
Dependencies: requires_one_of
Recommender: starter to complete
Pricing: price within catalogue range by term, volume, bundle
GST, setup, contract value

Orders and payments
Order states and event timeline
UPI, net banking, card gateway
Amount set on server, idempotent
HMAC-verified webhooks
Install slots, gated on pay + KYC

Smart KYC pipeline
1 Consent: purpose, retention
2 OCR on our server, deskew
3 Parse fields, repair GSTIN
4 GSTIN and Udyam lookup
5 Name and address matching
→ verified · needs_review · action_required · incomplete · rejected, each with the fix

Analytics and back-office
Funnel, segments, lead sources
Propensity model (AUC 0.68)
Unclosed advisor journeys become a call list
KYC review queue for admins

LLM provider (optional)
Gemini or Claude. Gets typed text, short history, catalogue facts.
Never documents or ID numbers.

Supplied catalogue
product_catalogue.csv (25 products)
pricing_rules.json (config)
read-only, seeds the products table

Payment gateway
Razorpay (production) or built-in sandbox. Card data never reaches or stays on our server.

Documents + registries
Encrypted files (Fernet), 30-day purge
Mock GSTIN registry: 3,534
Mock Udyam registry: 2,723

Supplied leads
smb_leads.csv: 5,000 leads
funnel, segments, attach rates, propensity model training

SQL database (SQLite default, PostgreSQL by URL): users · products · journeys · advisor messages · quotes · orders and events · KYC cases (identifiers encrypted). Every module persists here.

Data flows

1. Owner types or speaks; the browser sends the text, or a clip of 25 s or less for transcription.
2. Structured needs (wants, rejections, quantities, budget) go to the catalogue engine.
3. The engine's facts (products, prices, reasons) return to the advisor for wording.
4. Typed text, short history and catalogue facts go to the LLM; the guard checks its reply.
5. The frozen quote becomes an order with a unique id.
6. Payment: the amount is computed on the server; the gateway confirms by signed webhook.
7. The owner uploads four documents after consent; OCR, registry checks and matching run on our server.
8. A verified KYC verdict plus payment unlocks install slots and the tracker.
9. Leads, orders and journeys feed the admin funnel, propensity score and call list.